Lecture # 11

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Interprocess communication

Exploring Interprocess Communication

- Definition:
  - **IPC**: Transfer of data among processes
- Need for IPC:
  - Overcoming limitations in basic communication mechanisms
  - Enabling dynamic and real-time communication

Types of IPC

1. Mapped Memory
2. Pipes
3. FIFOs
4. Shared Memory
5. Sockets

Mapped Memory

- Mapped memory allows different processes to communicate through a shared file.
- It is distinct from shared memory segments and can be used for interprocess communication or accessing file contents.
- Mapped memory creates an association between a file and a process's memory, splitting the file into page-sized chunks.
- Linux copies these chunks into virtual memory pages, enabling fast access to file contents

Mapping an Ordinary File

- To map an ordinary file, use the mmap ("Memory MAPped") call.
- Parameters include the address to map, length in bytes, protection (read, write, execute), flags, file descriptor, and offset.
- Flag values include MAP_FIXED for a fixed address, MAP_PRIVATE for non-shared writes, and MAP_SHARED for immediate file reflection.
- Successful mmap returns a pointer to the beginning of memory; use munmap to release mapped memory.

Pipes

A core Unix principle is modeling things as files.

Could we use a "file"?

Idea: a file that one process could write, and another process could read?

Problem: we don't want to clutter the filesystem with actual files every time two processes want to communicate.

Solution: have the operating system set this up for us. It will give us two new file descriptors - one for writing, another for reading.

If someone writes data to the write FD, it can be read from the read FD. It's not actually a physical file on disk - we are just using files as an abstraction

pipe()

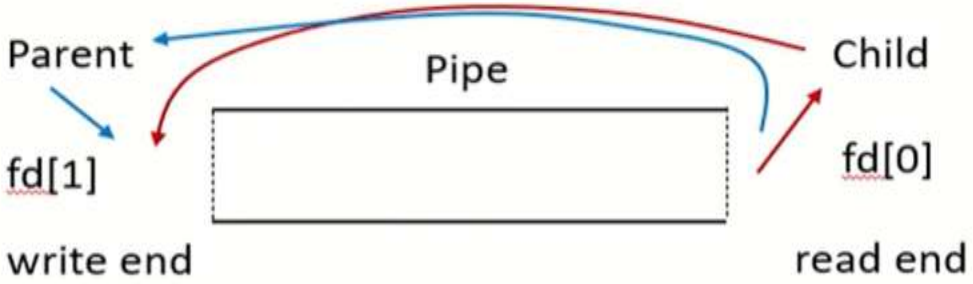
int pipe(int fds[]);

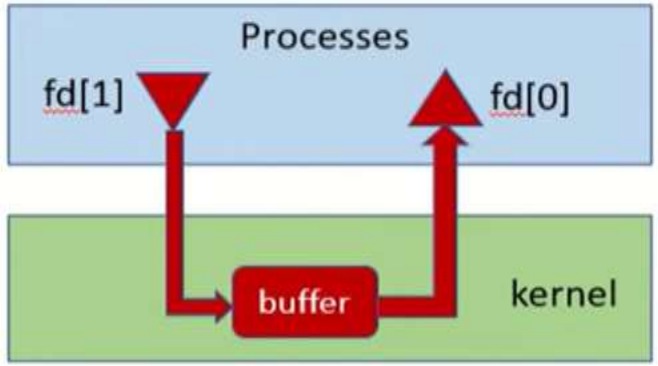
The pipe system call populates the 2-element array fds with two file descriptors such that everything written to fds[1]can be read from fds[0].

Returns 0 on success, or -1 on error.

IPC - Pipes

- Pipe is accessed through `int fd[]` file descriptors:
  - fd[0] is read end
  - fd[1] is write end
- UNIX treats pipe as a special type of file
  - Pipes can be accessed using `read()` and `write()` system calls





pipe() - example

```c
static const char * kPipeMessage = "Hello, this message is coming through a pipe.";
int main(int argc, char *argv[]) {
    int fds[2];
    int result = pipe(fds);

    // Write message to pipe (assuming here all bytes written immediately)
    write(fds[1], kPipeMessage, strlen(kPipeMessage) + 1);
    close(fds[1]);

    // Read message from pipe
    char receivedMessage[strlen(kPipeMessage) + 1];
    read(fds[0], receivedMessage, sizeof(receivedMessage));
    close(fds[0]);
    printf("Message read: %s\n", receivedMessage);

    return 0;
}
```

```
$ ./pipe-demo
Message read: Hello, this message is coming through a pipe.
```

Question

HOW CAN PIPE WORK FOR TWO PROCESSES?

pipe()

pipe can allow processes to communicate!

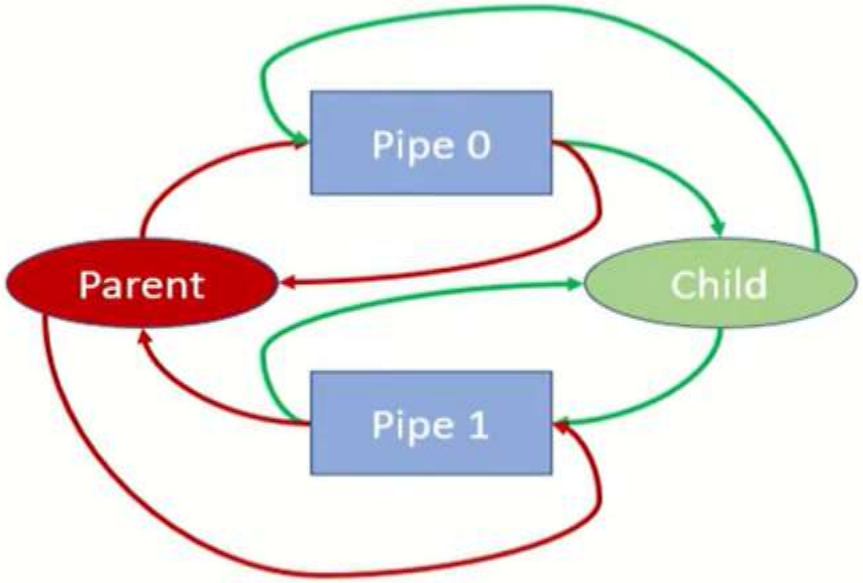
The parent's file descriptor table is replicated in the child - both have pipe access (increasing reference counts in open file table)

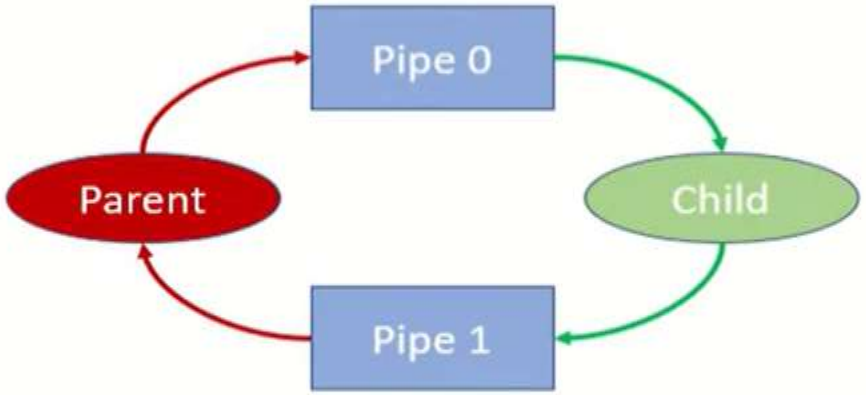
E.g. the parent can write to the "write" end and the child can read from the "read" end

Because they're file descriptors, there's no global name for the pipe (another process can't "connect" to the pipe).

Each pipe is uni-directional (one end is read, the other write)

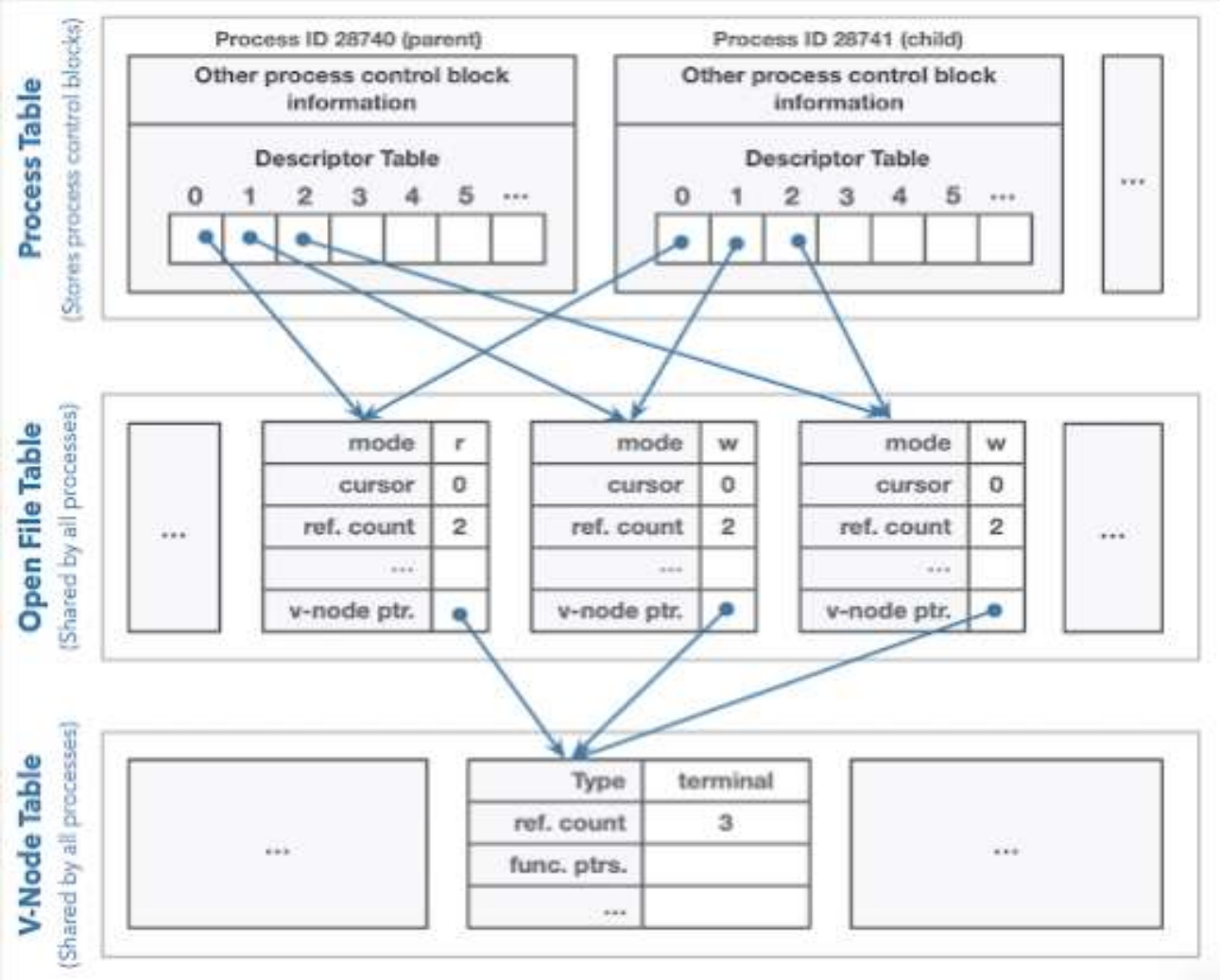
Two-way Communication



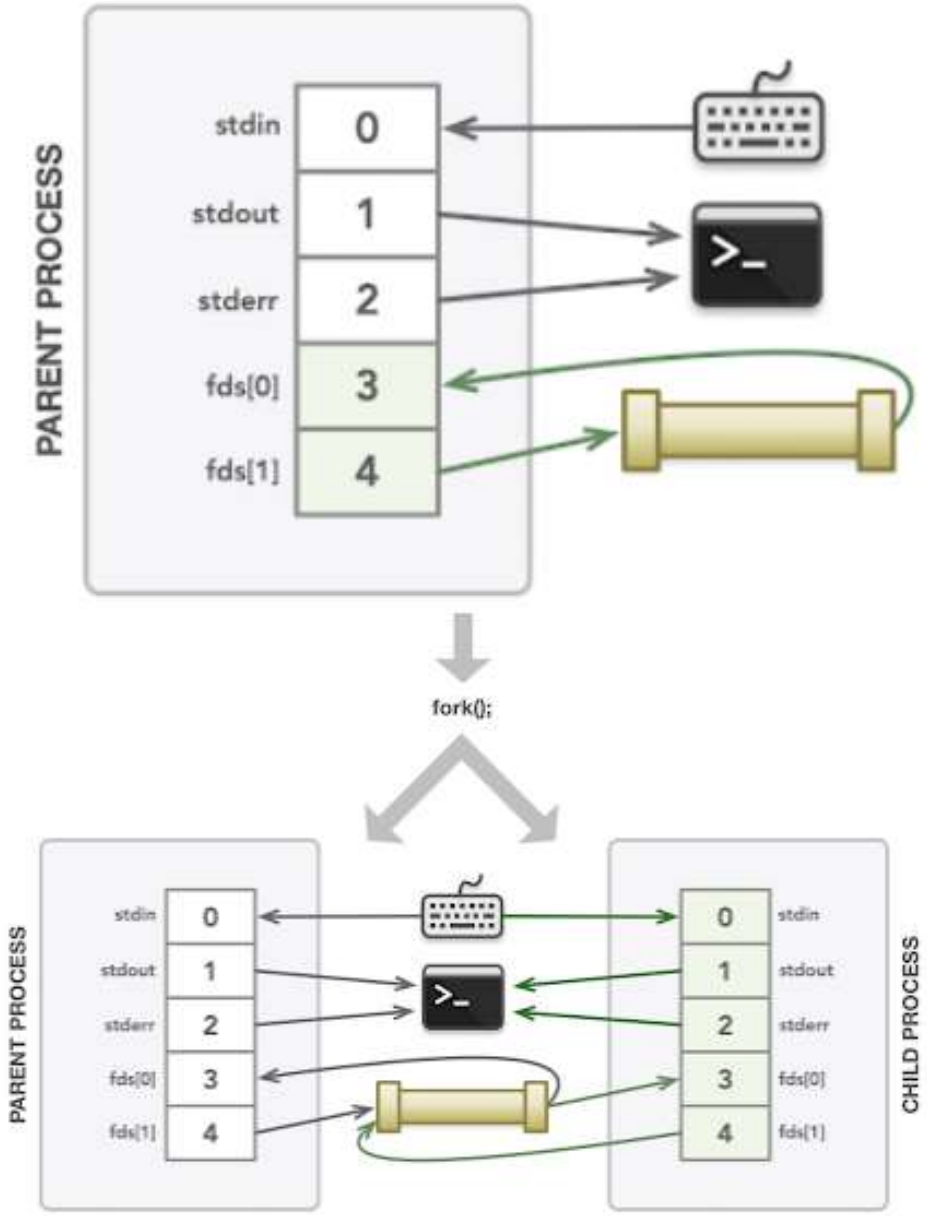


Close unnecessary ends

pipe()


Process Table
(Stores process control blocks)
Process ID 28740 (parent)
Other process control block information
Descriptor Table
0 1 2 3 4 5 ...
Process ID 28741 (child)
Other process control block information
Descriptor Table
0 1 2 3 4 5 ...
...
Open File Table
(Shared by all processes)
...
mode r
cursor 0
ref. count 2
...
v-node ptr.
mode w
cursor 0
ref. count 2
...
v-node ptr.
mode w
cursor 0
ref. count 2
...
v-node ptr.
...
V-Node Table
(Shared by all processes)
...
Type terminal
ref. count 3
func. ptrs.
...
...

Parent-Child Communication



Parent-Child Communication

```c
static const char * kPipeMessage = "Hello, this message is coming through a pipe.";
int main(int argc, char *argv[]) {
    int fds[2];
    pipe(fds);
    size_t bytesSent = strlen(kPipeMessage) + 1;

    pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        // In the child, we only read from the pipe
        close(fds[1]);
        char buffer[bytesSent];
        read(fds[0], buffer, sizeof(buffer));
        close(fds[0]);
        printf("Message from parent: %s\n", buffer);
        return 0;
    }

    // In the parent, we only write to the pipe (assume everything is written)
    close(fds[0]);
    write(fds[1], kPipeMessage, bytesSent);
    close(fds[1]);
    waitpid(pidOrZero, NULL, 0);
    return 0;
}
```

Example program

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {

    int fd[2], n;

    char buffer[100];

    pid_t p;


    pipe(fd); // creates a unidirectional pipe with
two ends fd[0] and fd[1]

    p = fork();
```

```
    if (p > 0) { // parent
        printf("Parent passing value to child\n");
        write(fd[1], "hello\n", 6); // fd[1] is the write end of the pipe
        close(fd[1]); // Close the write end since we're done writing
        wait(NULL);    // Pass NULL as an argument to wait to ignore
the exit status
    } else { // child
        close(fd[1]); // Close the write end since we're not using it
        printf("Child printing received value\n");
        n = read(fd[0], buffer, 100); // fd[0] is the read end of the
pipe
        write(1, buffer, n);
        close(fd[0]); // Close the read end since we're done reading
    }

    return 0;
}
```

Header Files:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

- These are standard C library and system header files. They provide functions related to input/output, process handling, and types.

Main Function:

```
int main() {
```

The program starts executing from the main function.

Variables and Pipe Creation:

```
int fd[2], n;
char buffer[100];
pid_t p;
pipe(fd);
```

fd[2]: An array to store file descriptors for the two ends of the pipe. fd[0] is the read end, and fd[1] is the write end.

- n: Variable to store the number of bytes read.
- buffer[100]: Character array to store the data read from the pipe.
- p: Process ID variable.

Pipe Creation:

pipe(fd);

Creates a unidirectional pipe with two ends (fd[0] for reading, fd[1] for writing).

Forking a Child Process:

p = fork();

Forks the process into a parent process and a child process. The value of p will be:

- Positive (p > 0) in the parent process, representing the child's PID.
- Zero (p == 0) in the child process.
- Negative (p < 0) if the fork fails.

Parent Process:

```
if (p > 0) {
   printf("Parent passing value to child\n");
   write(fd[1], "hello\n", 6);
   close(fd[1]);
   wait(NULL);
}
```

- The parent process writes the string "hello\n" to the write end of the pipe (fd[1]).
- The write end is then closed since the parent is done writing.
- The parent waits for the child process to finish execution using wait(NULL).
- the received value using write(1, buffer, n).

Child Process:

```
else {
   close(fd[1]);
   printf("Child printing received value\n");
   n = read(fd[0], buffer, 100);
   write(1, buffer, n);
   close(fd[0]);
}
```

- The child process closes the write end of the pipe (fd[1]) since it is not using it.
- It reads data from the read end of the pipe (fd[0]) into the buffer.
- The read end is then closed since the child is done reading.
- The child prints the received value using write(1, buffer, n).

Question

WHAT IS THE REFERENCE COUNT FOR THE PIPE(FILE) THROUGHOUT THE PROGRAM?

Question

WHAT ARE THE POSSIBLE BEHAVIOR OF THE READ COMMAND?

Key Ideas

Because the pipe file descriptors are duplicated in the child, we need to close the 2 pipe ends in both the parent and the child.

read() blocks until the bytes are available or there is no more to read (e.g. end of file or pipe write end closed). If the parent hasn't written yet, the child's call to read() will wait

Question

WHAT ABOUT BIDIRECTIONAL COMMUNICATION?

Demo

HTTPS://CPLAYGROUND.COM/?P=EAGLE-FISH-MOUSE

HTTPS://CPLAYGROUND.COM/?P=BEAR-CHAMELEON-MOSQUITO

pipe

This method of communication between processes relies on the fact that file descriptors are duplicated when forking.

- each process has its own copy of both file descriptors for the pipe
- both processes could read or write to the pipe if they wanted.
- each process must therefore close both file descriptors for the pipe when finished

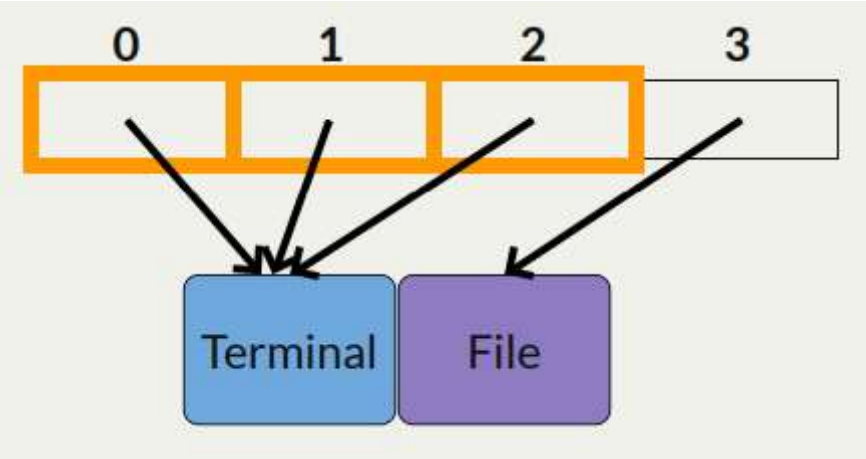
This is the core idea behind how a shell can support piping between processes

(e.g. cat file.txt | uniq | sort).

Redirecting Process I/O

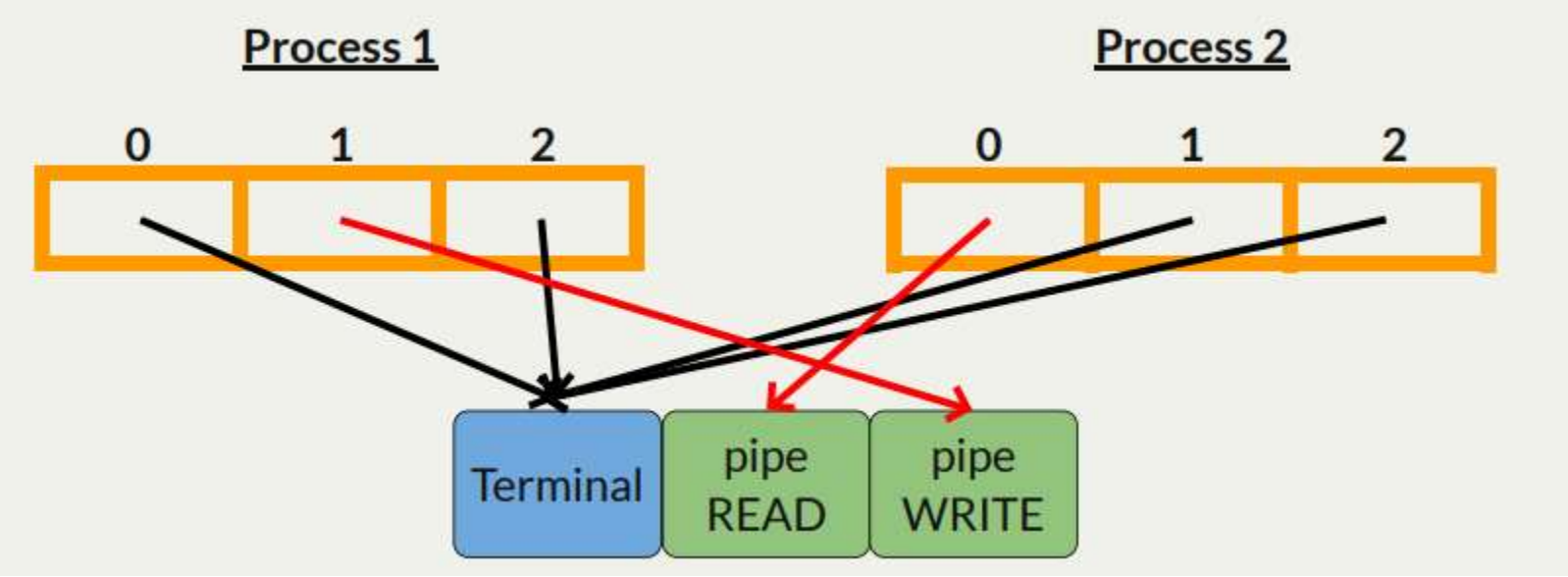
Each process has the special file descriptors STDIN (0), STDOUT (1) and STDERR (2)

Processes assume these indexes are for these methods of communication (e.g. printf always outputs to file descriptor 1, STDOUT).



Redirecting Process I/O

Could we do this with a pipe?



Why would this be useful?

Challenge

Implement “ls | wc -l” in C

Hint: you may require exec() and dup2()

Solution:
https://cplayground.com/?p=whale-octopus-jaguar

Example from Beej’s Guide to Interprocess Communication